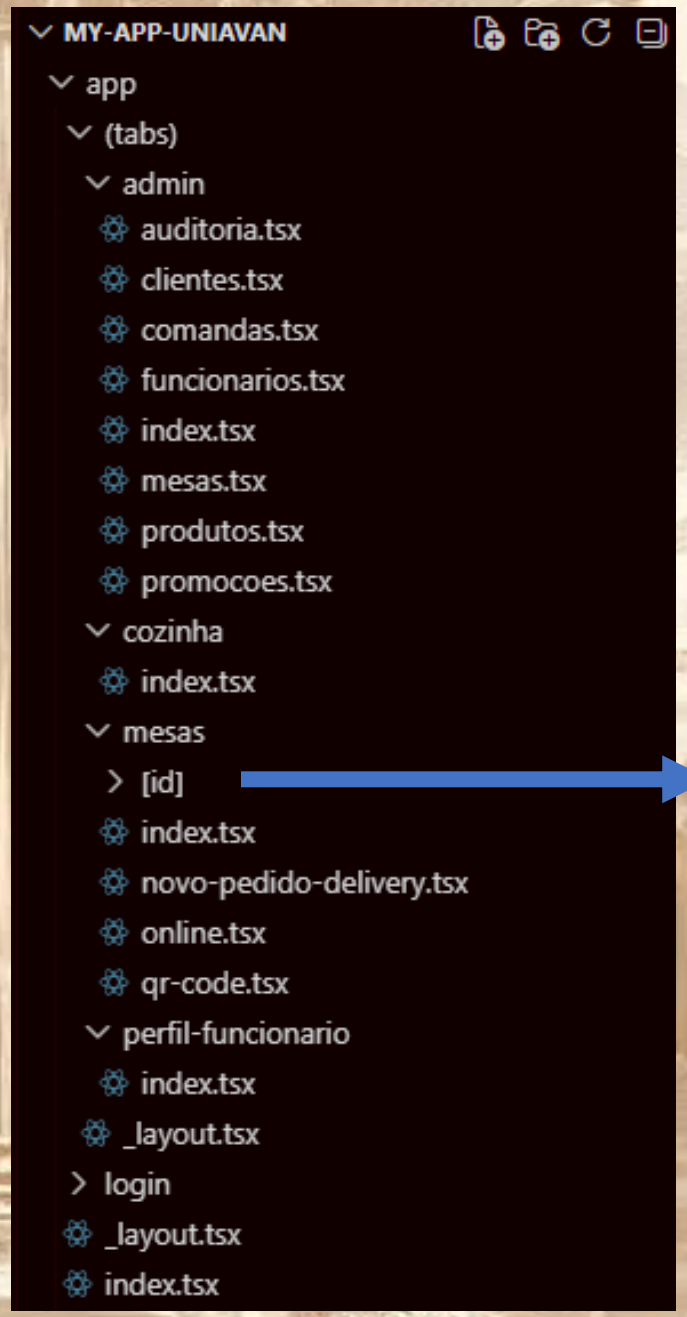
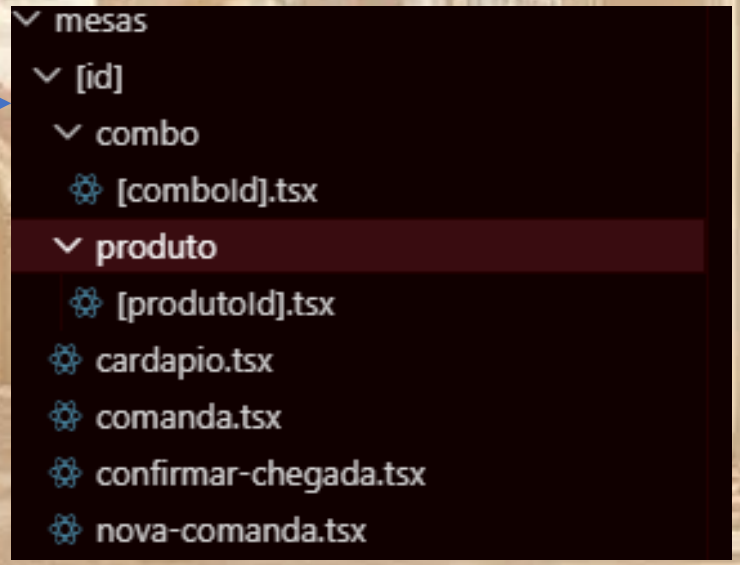
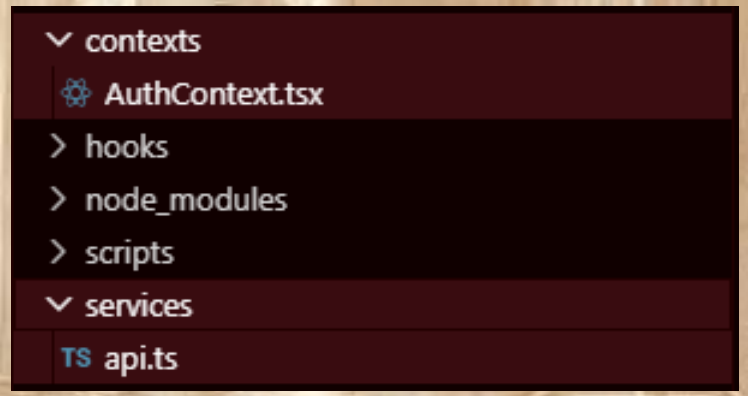
# Arquitetura do Sistema

# Organização do Projeto



_layout.tsx

app > _layout.tsx

```tsx
import { Stack } from 'expo-router';
import { AuthProvider, useAuth } from '../contexts/AuthContext';
import { ActivityIndicator, View } from 'react-native';

function RootLayout() {
  const { loading } = useAuth();

  if (loading) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
        <ActivityIndicator size="large" color="#8D0000" />
      </View>
    );
  }

  // Sempre renderiza o Stack; a tela inicial (app/index.tsx) fará o redirecionamento
  return <Stack screenOptions={{ headerShown: false }} />;
}

export default function Layout() {
  return (
    <AuthProvider>
      <RootLayout />
    </AuthProvider>
  );
}
```

index.tsx

app > index.tsx

```tsx
import { Redirect } from 'expo-router';
import { useAuth } from '../contexts/AuthContext';

export default function Index() {
  const { user} = useAuth();

  if (!user) {
    return <Redirect href="/login" />;
  }

  if (user.cargo === 'cozinha') {
    return <Redirect href="/(tabs)/cozinha" />;
  }

  if (user.cargo === 'admin' || user.cargo === 'gerente') {  // A especificar melhor, mas a priori já é suficiente
    return <Redirect href="/(tabs)/admin" />;
  }

  return <Redirect href="/(tabs)/mesas" />;
}

```


_layout.tsx
app > (tabs) > _layout.tsx
export default function TabsLayout() {
return (
>
<Tabs.Screen
name="cozinha/index"
options={{
title: 'Cozinha',
tabBarIcon: ({ color }) => <Feather name="clock" size={22} color={color} />,
href: isCozinha ? undefined : null,
}}
/>
<Tabs.Screen
name="mesas/index"
options={{
title: 'Mesas',
tabBarIcon: ({ color }) => <Feather name="grid" size={22} color={color} />,
href: isGarcom || isAdminOuGerente ? undefined : null,
}}
/>
<Tabs.Screen
name="admin/index"
options={{
title: 'Admin',
tabBarIcon: ({ color }) => <Feather name="shield" size={22} color={color} />,
href: isAdminOuGerente ? undefined : null,
}}
/>
<Tabs.Screen
name="perfil-funcionario/index"
options={{
title: 'Perfil',
tabBarIcon: ({ color }) => <Feather name="user" size={22} color={color} />,
}}
/>

TS api.ts

services > TS api.ts

```ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import { API_BASE_URL } from '../config';

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
});

api.interceptors.request.use(async (config) => {
  if (!config.headers.Authorization) {
    const token = await AsyncStorage.getItem('@access_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
  }
  return config;
});

export default api;
```

TS *config.ts* ×

TS config.ts

```ts
import Constants from 'expo-constants';

const getApiUrl = (): string => {
  const hostUri = Constants.manifest?.hostUri || Constants.expoConfig?.hostUri;
  if (hostUri) {
    const ip = hostUri.split(':')[0];
    return `http://${ip}:8000`;
  }

  return 'http://localhost:8000';
};

export const API_BASE_URL = getApiUrl();
```

AuthContext.tsx

contexts > AuthContext.tsx

```tsx
25
26  const AuthContext = createContext<AuthContextData>(
27    {} as AuthContextData
28  );
29
30  export const AuthProvider: React.FC<{
31    children: React.ReactNode;
32  }> = ({ children }) => {
33    const [user, setUser] = useState<User | null>(null);
34    const [loading, setLoading] = useState(true);
35
36    useEffect(() => {
37      loadStoredData();
38    }, []);
39
40    async function loadStoredData() {
41      try {
42        const token = await AsyncStorage.getItem('@access_token');
43        const userData = await AsyncStorage.getItem('@user_data');
44
45        if (token && userData) {
46          api.defaults.headers.common[
47            'Authorization'
48          ] = `Bearer ${token}`;
49
50          setUser(JSON.parse(userData));
51
52        }
53      } catch (error) {
54        console.error('Erro ao carregar sessão:', error);
55      } finally {
56        setLoading(false);
57      }
58    }
```

AuthContext.tsx

contexts > AuthContext.tsx

```tsx
32  }> = ({ children }) => {
60    async function signIn(email: string, password: string) {
61    try {
62      const formData = new URLSearchParams();
63      formData.append('username', email);
64      formData.append('password', password);
65
66      const response = await api.post('/auth/token', formData, {
67        headers: { 'Content-Type': 'application/
          x-www-form-urlencoded' },
68      });
69
70      const { access_token } = response.data;
71      if (!access_token) throw new Error('Token não recebido');
72
73      // Salva token ANTES de usar
74      await AsyncStorage.setItem('@access_token', access_token);
75
76      // Configura o header padrão
77      api.defaults.headers.common['Authorization'] = `Bearer $
        {access_token}`;
78
79      // Agora o interceptor encontrará o token no storage
80      const meResponse = await api.get('/funcionarios/me');
81      const userData = meResponse.data;
82
83      await AsyncStorage.setItem('@user_data', JSON.stringify
        (userData));
84      setUser(userData);
85
86      console.log('Login realizado com sucesso');
87    } catch (error: any) {
88      console.error('Erro no login:', error.response?.data || error.
        message);
```

index.tsx

app > login > index.tsx

```tsx
export default function LoginScreen() {
  const router = useRouter();
  const { signIn, loading: authLoading } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleLogin = async () => {
    if (!email.trim() || !password.trim()) {
      setError('Preencha e-mail e senha');
      return;
    }
    setError('');
    setLoading(true);
    try {
      await signIn(email.trim(), password.trim());
      router.replace('/(tabs)/mesas');
    } catch (err: any) {
      setError(err.message || 'Falha no login. Verifique suas credenciais.');
    } finally {
      setLoading(false);
    }
  };

  const handleForgotPassword = () => {
    router.push('/login/forgot-password');
  };

```

index.tsx

app > (tabs) > admin > index.tsx

```tsx
interface AdminCard {
  title: string;
  icon: keyof typeof Feather.glyphMap;
  route: string;
  color: string;
  desc: string;
}

const CARDS: AdminCard[] = [
  { title: 'Clientes', icon: 'users', route: '/(tabs)/admin/clientes', color: '#2A6B2A', desc: 'Gerenciar clientes cadastrados' },
  { title: 'Mesas', icon: 'grid', route: '/(tabs)/admin/mesas', color: '#8D0000', desc: 'Gerenciar mesas do salão' },
  { title: 'Produtos', icon: 'shopping-bag', route: '/(tabs)/admin/produtos', color: '#E68A00', desc: 'Gerenciar cardápio' },
  { title: 'Funcionários', icon: 'user-check', route: '/(tabs)/admin/funcionarios', color: '#7B5800', desc: 'Gerenciar equipe' },
  { title: 'Promoções', icon: 'tag', route: '/(tabs)/admin/promocoes', color: '#2A3406', desc: 'Cupons e descontos' },
  { title: 'Comandas', icon: 'list', route: '/(tabs)/admin/comandas', color: '#333', desc: 'Gerenciar comandas' },
  { title: 'Auditoria', icon: 'file-text', route: '/(tabs)/admin/auditoria', color: '#555', desc: 'Logs de auditoria' },
];

```

auditoria.tsx

app > (tabs) > admin > auditoria.tsx

```tsx
interface AuditLog {
  id: number;
  usuario_tipo: string;
  usuario_id: number | null;
  acao: string;
  tabela_afetada?: string;
  registro_id?: number;
  timestamp: string;
  ip?: string;
  user_agent?: string;
  dados_anteriores?: any;
  dados_novos?: any;
  funcionario_rel?: { nome: string } | null;
}

export default function AdminAuditoriaScreen() {
  const [logs, setLogs] = useState<AuditLog[]>([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);

  const fetchData = useCallback(async (showLoader = true) => {
    if (showLoader) setLoading(true);
    try {
      const { data } = await api.get<AuditLog[]>('/audit-logs/');
      setLogs(data);
    } catch {
      // silently fail
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  }, []);

```

auditoria.tsx | clientes.tsx

app > (tabs) > admin > clientes.tsx

```tsx
export default function AdminClientesScreen() {
  const fetchClientes = useCallback(async (showLoader = true) => {
    if (showLoader) setLoading(true);
    try {
      const params: Record<string, any> = { limite: 100 };
      if (busca.trim()) params.busca = busca.trim();
      const { data } = await api.get<Cliente[]>('/clientes/', { params });
      setClientes(data);
    } catch {
      Alert.alert('Erro', 'Não foi possível carregar clientes.');
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  }, [busca]);

  useFocusEffect(useCallback(() => { fetchClientes(); }, [fetchClientes]));

  const toggleAtivo = (cliente: Cliente) => {
    const acao = cliente.ativo ? 'desativar' : 'ativar';
    Alert.alert(`${acao === 'ativar' ? 'Ativar' : 'Desativar'} cliente`, `Tem certeza?`, [
      { text: 'Cancelar', style: 'cancel' },
      {
        text: 'Sim',
        onPress: async () => {
          try {
            await api.post(`/clientes/${cliente.id}/${acao}`);
            fetchClientes(false);
          } catch {
            Alert.alert('Erro', `Falha ao ${acao} cliente.`);
          }
        },
      },
    ]);
```

comandas.tsx

app > (tabs) > admin > comandas.tsx

```tsx
export default function AdminComandasScreen() {
  const baixarPdf = async () => {
    if (!reciboComanda) return;
    setGerandoPdf(true);
    try {
      const html = gerarHtmlRecibo(reciboComanda);
      const { uri } = await Print.printToFileAsync({ html, base64: false });
      if (await Sharing.isAvailableAsync()) {
        await Sharing.shareAsync(uri, { mimeType: 'application/pdf', dialogTitle: `Recibo #${reciboComanda.id}` });
      } else {
        Alert.alert('PDF salvo', `Arquivo: ${uri}`);
      }
    } catch {
      Alert.alert('Erro', 'Não foi possível gerar o PDF.');
    } finally { setGerandoPdf(false); }
  };

  const enviarEmail = async () => {
    if (!reciboComanda) return;
    try {
      await api.post(`/comandas/${reciboComanda.id}/enviar-recibo`);
      Alert.alert('Sucesso', 'Recibo enviado por e-mail.');
    } catch (err: any) {
      const msg = err.response?.data?.detail || 'Erro ao enviar e-mail.';
      Alert.alert('Erro', msg);
    }
  };

```

produtos.tsx

app > (tabs) > admin > produtos.tsx

```tsx
30  export default function AdminProdutosScreen() {
84    const salvar = async () => {
85      if (!nome.trim() || !preco.trim()) {
86        Alert.alert('Erro', 'Nome e preço são obrigatórios.');
87        return;
88      }
89      setSalvando(true);
90      const payload = {
91        nome: nome.trim(),
92        descricao: descricao.trim(),
93        imagem_link: imagem.trim() || 'https://via.placeholder.com/200',
94        preco: parseFloat(preco.replace(',', '.')),
95        id_categoria: categoriaId || categorias[0]?.id,
96        tempo_preparo_medio: tempoPreparo ? parseInt(tempoPreparo, 10) : null,
97        popular,
98        disponivel,
99      };
100     try {
101       if (editando) {
102         await api.put(`/produtos/${editando.id}`, payload);
103       } else {
104         await api.post('/produtos/', payload);
105       }
106       setModalVisible(false);
107       fetchData();
108     } catch (err: any) {
109       Alert.alert('Erro', err.response?.data?.detail || 'Falha ao salvar.');
110     } finally {
111       setSalvando(false);
112     }
113   };
```

index.tsx

app > (tabs) > cozinha > index.tsx

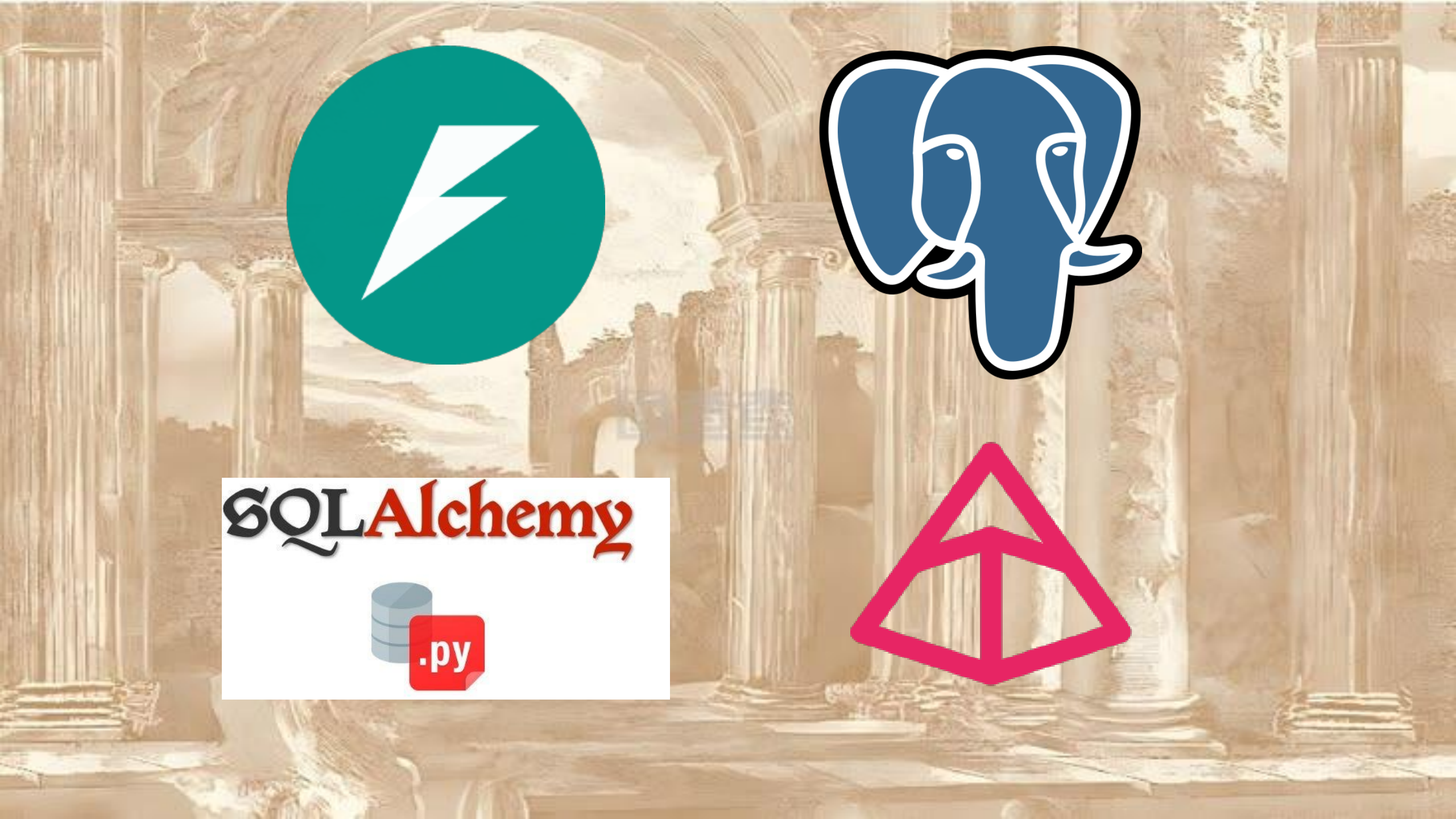
```tsx
export default function CozinhaScreen() {

  const atualizarStatus = async (comandaId: number, novoStatus: string) => {
    try {
      await api.post(`/comandas/${comandaId}/status`, { status_novo: novoStatus });
      fetchComandas(false);
    } catch (err: any) {
      Alert.alert('Erro', err.response?.data?.detail || 'Falha ao atualizar status.');
    }
  };

  const confirmarAcao = (comandaId: number, acao: string, label: string) => {
    Alert.alert(
      `Confirmar`,
      `Marcar como "${label}"?`,
      [
        { text: 'Cancelar', style: 'cancel' },
        { text: 'Sim', onPress: () => atualizarStatus(comandaId, acao) },
      ]
    );
  };
```

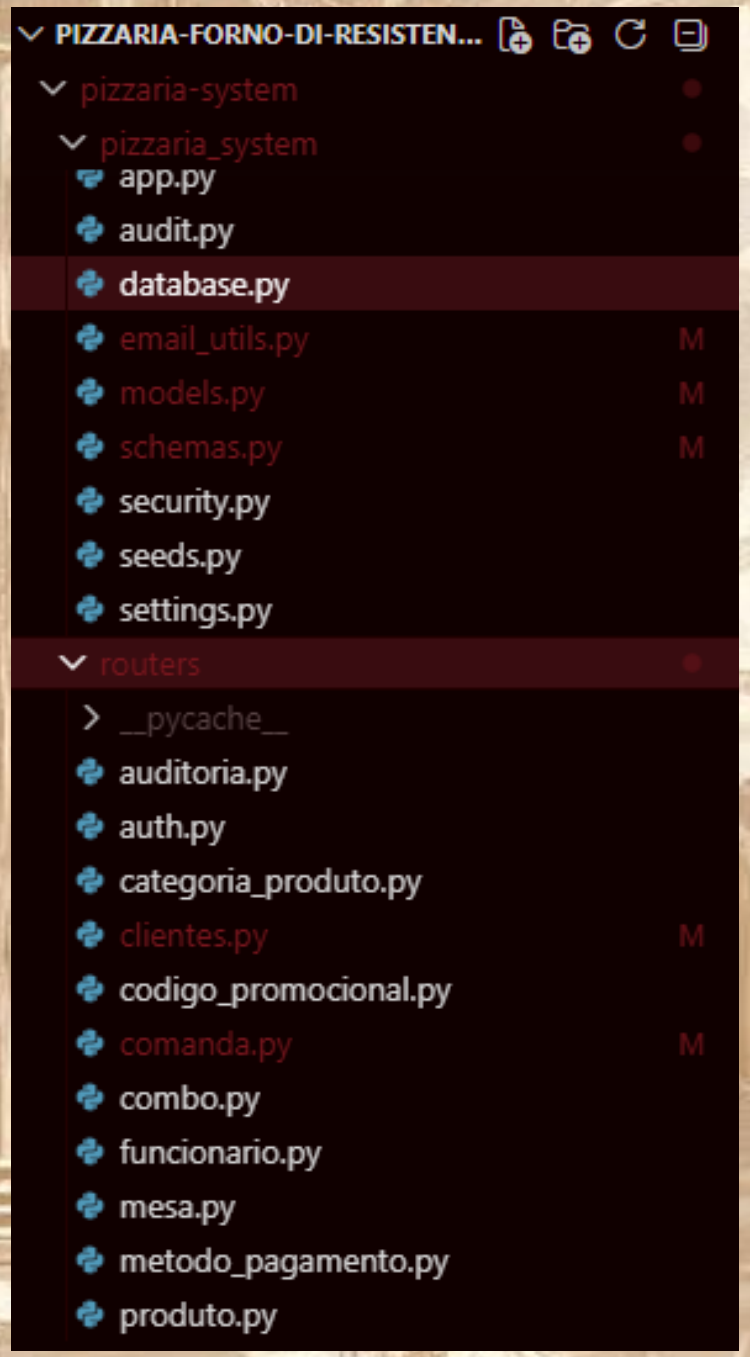

[produtoId].tsx
app > (tabs) > mesas > [id] > produto > [produtoId].tsx
21 export default function ProdutoDetalheScreen() {
40 const handleAdicionar = async () => {
41 if (!produto) return;
42
43 // Se temos comandaId, adiciona diretamente à comanda existente
44 if (comandaId) {
45 try {
46 await api.post(`/comandas/${comandaId}/itens`, {
47 id_produto: produto.id,
48 id_combo: undefined,
49 quantidade,
50 observacao: observacao.trim() || null,
51 });
52 Alert.alert('Sucesso', 'Item adicionado à comanda!');
53 router.replace(`/(tabs)/mesas/${mesaId}/comanda`);
54 } catch (err: any) {
55 Alert.alert('Erro', err.response?.data?.detail || 'Falha ao adicionar item.');
56 }
57 } else {
58 // Comportamento original: navega para nova-comanda
59 router.push({
60 pathname: `/(tabs)/mesas/${mesaId}/nova-comanda`,
61 params: {
62 tipo: 'produto',
63 itemId: String(produto.id),
64 nome: produto.nome,
65 preco: String(produto.preco),
66 quantidade: String(quantidade),
67 observacao,
68 },
69 });
70 }
71 };

cardapio.tsx

app > (tabs) > mesas > [id] > cardapio.tsx

```tsx
export default function CardapioScreen() {
  // Handlers de navegação / adição
  const handleItemPress = (item: ItemCardapio) => {
    const basePath = `/(tabs)/mesas/${mesaId}`;
    if (comandaId) {
        // Navega para a tela de detalhe, passando comandaId como parâmetro extra
        if (item.tipo === 'produto') {
        router.push({
            pathname: `${basePath}/produto/${item.id}`,
            params: { comandaId }
        });
        } else {
        router.push({
            pathname: `${basePath}/combo/${item.id}`,
            params: { comandaId }
        });
        }
    } else {
        // Fluxo original sem comanda ativa
        if (item.tipo === 'produto') {
        router.push(`${basePath}/produto/${item.id}`);
        } else {
        router.push(`${basePath}/combo/${item.id}`);
        }
    }
    };
```


SQLAlchemy
.py




```
============================================================ tests coverage ============================
==============================
_____________________________________________ coverage: platform win32, python 3.11.9-final-0 __
______________________________

Name                                    Stmts   Miss  Cover
-----------------------------------------------------------
pizzaria_system\__init__.py                 0      0   100%
pizzaria_system\app.py                     25      3    88%
pizzaria_system\audit.py                   18      0   100%
pizzaria_system\database.py                11      4    64%
pizzaria_system\email_utils.py             88     74    16%
pizzaria_system\models.py                 195      1    99%
pizzaria_system\schemas.py                382      0   100%
pizzaria_system\security.py                44      2    95%
pizzaria_system\seeds.py                   16     11    31%
pizzaria_system\settings.py                15      0   100%
-----------------------------------------------------------
TOTAL                                     794     95    88%
============================================================ 255 passed in 433.96s (0:07:13) ==========
```

security.py

pizzaria-system > pizzaria_system > security.py > ...

```python
def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)


def verify_password_hash(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)


def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(tz=ZoneInfo('UTC')) + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({'exp': expire})
    encoded_jwt = encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
    return encoded_jwt

```